



Tutaj masz link do gry:

https://games.cdn.famobi.com/html5games/c/cannon-balls-3d/v240/?fg_domain=play.famobi.com&fg_aid=A1000-1&fg_uid=3ffa11c2-61da-4974-892b-473582253371&fg_pid=4638e320-4444-4514-81c4-d80a8c662371&fg_beat=866&original_ref=https%3A%2F%2Fgames.cdn.famobi.com%2F

Zbadaj ją, rozpisz mechanikę, fizykę, zależności i jak działa.
Rozpisz specyfikację techniczną do zbudowania podobnej gry.

Przeanalizowałem strukturę tej gry na poziomie runtime i asset loadera: to HTML5/WebGL zbudowane w PlayCanvas, z preloadem fizyki Ammo/WebAssembly, pojedynczym canvasem fullscreen i zestawem skryptów ładowanych dynamicznie przed startem sceny.

Na tej podstawie da się dość precyzyjnie rozpisać, jak działa pętla gry, system strzału, fizyka destrukcji i co trzeba zbudować, żeby odtworzyć podobny produkt.

Architektura gry

Strona gry ładuje `famobi-game-interface.js`, a następnie inicjalizuje silnik przez listę plików: `pako.min.js`, `playcanvas-stable.min.js`, `playcanvas-config-loader-patch.js`, `APIMediator.js`, `__settings__.js`, `__modules__.js`, `__start__.js` i `__loading__.js`, co wskazuje na standardowy bootstrap PlayCanvas z opakowaniem portalowym Famobi.

W konfiguracji gry ustawiono scenę startową `2340452.json`, wejście z myszy i dotyku oraz preload modułu Ammo z plików `ammo.wasm.js` i `ammo.wasm.wasm`, więc fizyka jest oparta o `rigidbody/collision` i działa natywnie przez WebAssembly.

Jak gra działa

To jest gra typu **physics puzzler**: gracz oddaje ograniczoną liczbę strzałów kulą z armaty, a celem jest przewrócenie lub zniszczenie konstrukcji z bloków i trafienie obiektów celu, zwykle przy możliwie małej liczbie pocisków.

Z uwagi na użycie PlayCanvas + Ammo, najbardziej prawdopodobny model to: kula jest instancjonowanym `rigidbody`, startuje z pozycji wylotu lufy, dostaje impuls lub `velocity` zgodne z kierunkiem celowania, a wynik poziomu zależy od kolizji, przewróceń, energii uderzenia i stanu obiektów oznaczonych jako cele.

Mechanika i fizyka

Typowy przebieg jednej próby w tej architekturze wygląda tak:

- Input: przytrzymanie lub drag ustawia kierunek i/lub siłę strzału, click/tap odpala kulę.
- Spawn: system tworzy prefab pocisku przy `muzzle point`, z `colliderem` i `rigidbody` dynamicznym.

- **Launch:** pocisk dostaje impuls liniowy, czasem z lekkim limitem maksymalnej prędkości i bez ciągłego sterowania po wystrzale.
- **Impact:** Ammo liczy kolizje z elementami wież, platform i triggerami celu; obiekty lekkie lub źle podparte wpadają w ruch i przewracają resztę układu.
- **Resolution:** poziom kończy się, gdy wszystkie obiekty celu znikną, spadną poniżej progu lub zostaną oznaczone jako zniszczone; porażka następuje po wyczerpaniu amunicji bez spełnienia warunku.

Fizyka takiej gry zwykle opiera się nie na „zniszczeniu mesh”, tylko na zależnościach stabilności: masa, środek ciężkości, tarcie, restitution, punkty podparcia i reakcja łańcuchowa po pierwszym trafieniu.

Najważniejsza zależność projektowa brzmi: gracz nie niszczy poziomu „damage systemem”, tylko destabilizuje konstrukcję, a satysfakcja bierze się z przewidywania, który klocek jest nośny i gdzie opłaca się oddać strzał.

Zależności systemowe

Żeby taka gra działała dobrze, systemy muszą być spięte w kilka warstw:

- **Ballistics:** kierunek, siła, cooldown, liczba pocisków, reset po locie poza mapę.
- **Physics scene:** rigidbody dynamic/static/kinematic, collidery prostych brył, grawitacja, sleep/awake.
- **Level logic:** definicja celów, stref out-of-bounds, licznik obiektów do zbiccia, sekwencja wygrana/przegrana.
- **Camera & UX:** kamera śledzi obszar akcji, ale zwykle nie komplikuje sterowania; UI pokazuje zapas kul i status poziomu.
- **Portal wrapper:** manifest PWA, preload splash, integracja językowa i metryki przez `APIMediator.js` oraz interfejs Famobi.

W praktyce najważniejsze są trzy relacje:

1. siła strzału vs masa pocisku,
2. masa elementów wieży vs ich podparcie,
3. warunek zaliczenia vs rzeczywisty stan fizyczny obiektów celu.

Jeśli te trzy warstwy są źle skalibrowane, gra robi się albo losowa, albo zbyt łatwa, bo jeden strzał rozwala wszystko niezależnie od punktu trafienia.

Specyfikacja techniczna

Poniżej gotowa specyfikacja do budowy podobnej gry w nowoczesnej wersji webowej:

Obszar	Specyfikacja
Silnik	PlayCanvas lub Three.js + Rapier/Ammo; w oryginale użyto PlayCanvas + Ammo WASM.
Render	WebGL2 z fallbackiem do WebGL1, fullscreen canvas, antialias włączony, alpha wyłączone.

Obszar	Specyfikacja
Fizyka	3D rigidbody, box/sphere colliders, fixed timestep 60 Hz, gravity ok. -9.81 m/s^2 jako punkt wyjścia projektowego.
Input	Mouse + touch, aim by drag, fire on release/tap; brak wymogu gamepada.
Kamera	Perspektywiczna, lekki follow/lerp na obszar kolizji, szybki reset po zakończeniu próby.
Pocisk	Sphere collider, dynamic rigidbody, spawn z lufy, impulse launch, despawn po czasie lub poza mapą.
Elementy poziomu	Static ground, dynamic blocks, target entities, trigger volumes, kill zone, checkpoint logic.
Win/Lose	Win po usunięciu wszystkich targetów; lose po wyczerpaniu amunicji i settled physics.
UI	Ekran ładowania, licznik kul, numer poziomu, retry, next level, win/lose overlay.
Dane poziomów	JSON lub Scriptable Objects: pozycje, rotacje, typy brył, masa, cel, limit strzałów, score thresholds.
Audio	Fire, impact light/heavy, collapse rumble, UI taps, win/lose stingers.
Analityka/portal	Adapter eventów: level_start, shot_fired, level_fail, level_complete, ad_break hooks.

Proponowany model obiektowy:

- GameManager — stany gry, poziomy, retry, przejścia.
- LevelManager — ładowanie układu i targetów.
- CannonController — aim, constraints kąta, fire.
- ProjectileController — spawn, lifetime, kontakt, trail.
- PhysicsMonitor — settled state, out-of-bounds, sleep check.
- TargetController — alive/down/destroyed.
- UIManager — HUD i ekrany końca poziomu.

Do zbudowania podobnej gry polecałbym taką implementację gameplayową:

1. Każdy poziom opisany jako JSON z prefabami bloków i targetów.
2. Armatę obracaj tylko w ograniczonym zakresie osi, żeby sterowanie było proste.
3. Strzał realizuj przez applyImpulse, nie przez ręczne przesuwanie pocisku.
4. Win condition opieraj o `remainingTargets == 0`, nie o sam kontakt pocisku.
5. Fail odpalaj dopiero wtedy, gdy wszystkie rigidbody „zasną” i nie ma już amunicji.

Jeśli chcesz, następnym krokiem mogę rozpisać to jeszcze w formie **pełnego GDD + technical design document**, z klasami, eventami, parametrami fizyki i strukturą level JSON pod implementację w PlayCanvas albo Three.js.